

Informe Laboratorio 4

Sección 1

Alumno Ariel Cáceres
e-mail: ariel.caceres@mail.udp.cl

Mayo de 2026

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	3
2.1. Investiga y documenta los tamaños de clave e IV	3
2.2. Solicita datos de entrada desde la terminal	3
2.3. Valida y ajusta la clave según el algoritmo	4
2.4. Implementa el cifrado y descifrado en modo CBC	5
2.5. Compara los resultados con un servicio de cifrado online	6
2.6. Describe la aplicabilidad del cifrado simétrico en la vida real	8
3. Conclusiones y Comentarios	8

1. Descripción de actividades

Desarrollar un programa en Python utilizando la librería pycrypto para cifrar y descifrar mensajes con los algoritmos DES, AES-256 y 3DES, permitiendo la entrada de la key, vector de inicialización y el texto a cifrar desde la terminal. Instrucciones:

1. Investigación
 - Investigue y documente el tamaño en bytes de la clave y el vector de inicialización (IV) requeridos para los algoritmos DES, AES-256 y 3DES. Mencione las principales diferencias entre cada algoritmo, sea breve.
2. El programa debe solicitar al usuario los siguientes datos desde la terminal
 - Key correspondiente a cada algoritmo.
 - Vector de Inicialización (IV) para cada algoritmo.
 - Texto a cifrar.
3. Validación y ajuste de la clave
 - Si la clave ingresada es menor que el tamaño necesario para el algoritmo complete los bytes faltantes agregando bytes adicionales generados de manera aleatoria (utiliza `get_random_bytes`).
 - Si la clave ingresada es mayor que el tamaño requerido, trunque la clave a la longitud necesaria.
 - Imprima la clave final utilizada para cada algoritmo después de los ajustes.
4. Cifrado y Descifrado
 - Implemente una función para cada algoritmo de cifrado y descifrado (DES, AES-256, y 3DES). Use el modo CBC para todos los algoritmos.
 - Asegúrese de utilizar el IV proporcionado por el usuario para el proceso de cifrado y descifrado.
 - Imprima tanto el texto cifrado como el texto descifrado.
5. Comparación con un servicio de cifrado online
 - Selecciona uno de los tres algoritmos (DES, AES-256 o 3DES), ingrese el mismo texto, key y vector de inicialización en una página web de cifrado online.
 - Compare los resultados de tu programa con los del servicio online. Valide si el resultado es el mismo y fundamente su respuesta.
6. Aplicabilidad en la vida real

- Describa un caso, situación o problema donde usaría cifrado simétrico. Defina que algoritmo de cifrado simétrico recomendaría justificando su respuesta.
- Suponga que la recomendación que usted entregó no fue bien percibida por su contraparte y le pide implementar hashes en vez de cifrado simétrico. Argumente cuál sería su respuesta frente a dicha solicitud.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Investiga y documenta los tamaños de clave e IV

A continuación, se tabulan los tamaños de clave y vectores de inicialización (IV) requeridos para el correcto funcionamiento de los algoritmos simétricos evaluados en modo de operación CBC (Cipher Block Chaining):

Algoritmo	Tamaño de Clave	Tamaño del IV	Estado de Seguridad
DES	8 bytes (64 bits)	8 bytes (64 bits)	Inseguro / Obsoleto
3DES	24 bytes (192 bits)	8 bytes (64 bits)	Deprecado / Inseguro
AES-256	32 bytes (256 bits)	16 bytes (128 bits)	Estándar Seguro Moderno

Tabla 1: Parámetros de entrada para algoritmos de bloques simétricos.

Las principales diferencias entre los algoritmos se resumen a continuación:

- **Estructura Interna:** DES y 3DES utilizan una Red de Feistel para sus operaciones iterativas. AES-256 rompe este paradigma utilizando una Red de Sustitución-Permutación (SPN), lo que permite operaciones en paralelo mucho más eficientes y mayor difusión de bits por ronda.
- **Fortaleza del Espacio de Claves:** DES está limitado a solo 56 bits efectivos de seguridad (8 bits se usan para paridad), haciéndolo vulnerable a ataques de fuerza bruta en minutos. 3DES mitiga esto aplicando DES tres veces consecutivas para un tamaño total de clave de 192 bits (168 bits efectivos), mientras que AES-256 proporciona un espacio de claves óptimo e infranqueable de 2^{256} combinaciones posibles.

Como se verá, en el código implementado se permiten tamaños de clave e IV fuera del tamaño requerido para el algoritmo seleccionado, truncando si supera el tamaño y rellenando con bytes aleatorios si son inferiores.

2.2. Solicita datos de entrada desde la terminal

Se diseñó una interfaz interactiva mediante la terminal en Python que despliega un menú numérico para seleccionar de forma independiente el algoritmo criptográfico que se desea

utilizar. Posteriormente, el programa solicita la entrada del texto plano a procesar, la palabra clave base y el vector de inicialización (IV), sugiriendo los tamaños recomendados de caracteres para cada algoritmo, como se evidencia en la figura 1.

```
(.venv) ariel@fedora:~/test/Lab4-cripto$ python3 lab4_crypto.py
***** LABORATORIO 4 DE CRIPTOGRAFÍA: CIFRADO SIMÉTRICO CBC *****
1. Probar DES
2. Probar 3DES
3. Probar AES-256
4. Salir
[+] Seleccione una opción (1-4): 3

*** CONFIGURACIÓN PARA AES-256 ***
[+] Ingrese el texto a cifrar: Hola mundo
[+] Ingrese la Clave (Recomendado 32 caracteres): clavearellenar
[+] Ingrese el IV (Recomendado 16 caracteres): IV-1234
```

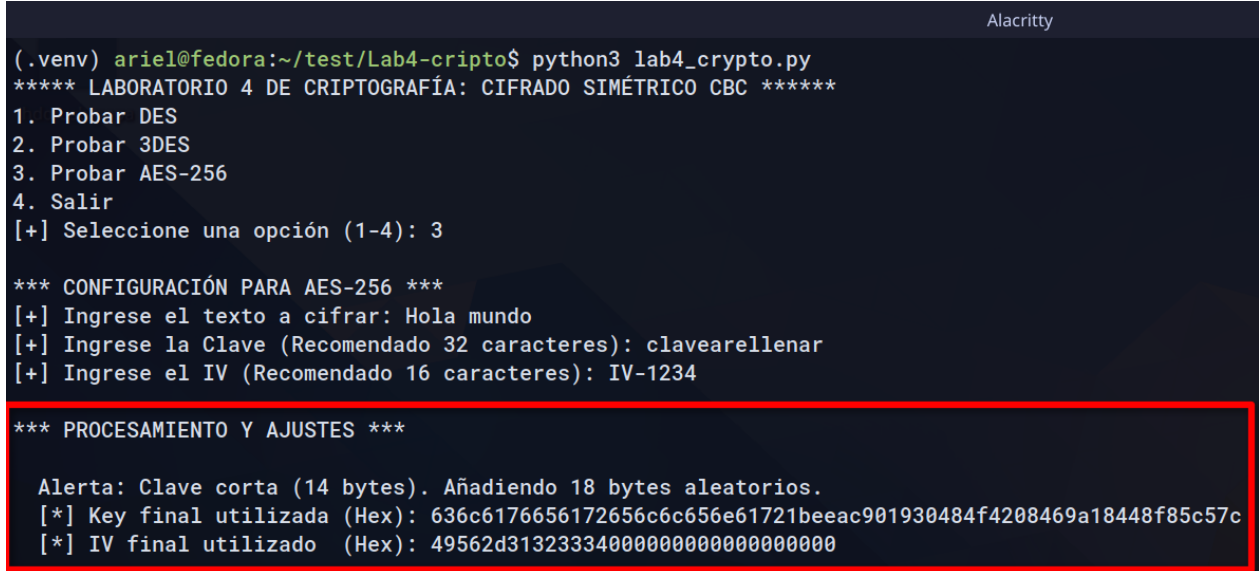
Figura 1: Solicitud de datos por terminal y despliegue de las claves finales ajustadas en formato Hexadecimal.

2.3. Valida y ajusta la clave según el algoritmo

Dado que el usuario puede ingresar claves de longitudes arbitrarias por consola, el script incluye un módulo de validación dinámica que fuerza la clave a los requisitos exactos de la especificación del algoritmo seleccionado:

- **Claves Cortas:** Si la longitud ingresada es inferior a la requerida, se completa el tamaño inyectando bytes aleatorios de alta entropía utilizando la función criptográficamente segura `get_random_bytes` de la librería **PyCryptodome** utilizada, ya que la librería **PyCrypto** sugerida se encuentra deprecada.
- **Claves Largas:** Si el texto de la clave supera el tamaño exigido, el script la trunca cortándola en el offset correspondiente.

Posteriormente, el programa imprime la clave final adaptada en formato hexadecimal para auditoría del usuario, como se observa en la figura 2.



```
Alacrity
(.venv) ariel@fedora:~/test/Lab4-cripto$ python3 lab4_crypto.py
***** LABORATORIO 4 DE CRIPTOGRAFÍA: CIFRADO SIMÉTRICO CBC *****
1. Probar DES
2. Probar 3DES
3. Probar AES-256
4. Salir
[+] Seleccione una opción (1-4): 3

*** CONFIGURACIÓN PARA AES-256 ***
[+] Ingrese el texto a cifrar: Hola mundo
[+] Ingrese la Clave (Recomendado 32 caracteres): clavearellenar
[+] Ingrese el IV (Recomendado 16 caracteres): IV-1234

*** PROCESAMIENTO Y AJUSTES ***

Alerta: Clave corta (14 bytes). Añadiendo 18 bytes aleatorios.
[*] Key final utilizada (Hex): 636c6176656172656c6c656e61721beeac901930484f4208469a18448f85c57c
[*] IV final utilizado (Hex): 49562d313233340000000000000000
```

Figura 2: Visualización de advertencias de ajuste y clave hexadecimal final.

2.4. Implementa el cifrado y descifrado en modo CBC

Para dar cumplimiento a los estándares de cifradores de bloques en modo CBC (Cipher Block Chaining), se estructuraron funciones dedicadas para cada algoritmo. Dado que el texto de entrada debe ser múltiplo exacto del tamaño del bloque del algoritmo, se implementó el esquema de relleno estándar (padding) PKCS#7 antes del cifrado, removiéndolo simétricamente tras descifrar.

El criptograma resultante (un arreglo de bytes binarios puros) es codificado a una cadena Base64 legible para facilitar su manipulación en la terminal en pantalla y comparaciones posteriores, como se observa en la figura 3. En el descifrado, debido a que el modo CBC almacena estados lógicos internos del bloque anterior, se inicializa un objeto de cifrado totalmente nuevo con la clave y el IV originales para recuperar el texto plano de forma exacta.

```

Alacritty
(.venv) ariel@fedora:~/test/Lab4-cripto$ python3 lab4_crypto.py
***** LABORATORIO 4 DE CRIPTOGRAFÍA: CIFRADO SIMÉTRICO CBC *****
1. Probar DES
2. Probar 3DES
3. Probar AES-256
4. Salir
[+] Seleccione una opción (1-4): 3

*** CONFIGURACIÓN PARA AES-256 ***
[+] Ingrese el texto a cifrar: Hola mundo
[+] Ingrese la Clave (Recomendado 32 caracteres): clavearellenar
[+] Ingrese el IV (Recomendado 16 caracteres): IV-1234

*** PROCESAMIENTO Y AJUSTES ***

Alerta: Clave corta (14 bytes). Añadiendo 18 bytes aleatorios.
[*] Key final utilizada (Hex): 636c6176656172656c6c656e61721beeac901930484f4208469a18448f85c57c
[*] IV final utilizado (Hex): 49562d313233340000000000000000

-----
[✓] Texto Cifrado (Base64): KYy67ShWjnCL9cGZ+P77cg==
[✓] Texto Descifrado:      Hola mundo
-----
(.venv) ariel@fedora:~/test/Lab4-cripto$ _

```

Figura 3: Output del criptograma en Base64 y reversibilidad del texto plano original.

2.5. Compara los resultados con un servicio de cifrado online

Se valida externamente la consistencia de la implementación mediante una comparación en la plataforma web de referencia *anycrypt.com/crypto*. Se seleccionó el estándar moderno **AES-256** bajo el modo de operación **CBC** y se introdujeron los mismos parámetros ingresados al script vistos en la figura 4:

- **Texto plano a cifrar:** ''Input textual para AES 256''
- **Clave de cifrado (Key):** ''Llavede32caracterespara256bitsxd'' (32 bytes exactos).
- **Vector de Inicialización (IV):** ''IVde16caracteres'' (16 bytes exactos).

Tanto el script desarrollado en Python como la interfaz de la página web devolvieron exactamente el mismo texto cifrado en Base64, confirmando la perfecta coincidencia del padding PKCS#7 y la integridad lógica de las operaciones criptográficas del código local realizado con la librería **PyCryptodome**. Comparación evidenciada en las figuras 4 y 5.

```

Alacrity
(.venv) ariel@fedora:~/test/Lab4-crypto$ python3 lab4_crypto.py
***** LABORATORIO 4 DE CRIPTOGRAFÍA: CIFRADO SIMÉTRICO CBC *****
1. Probar DES
2. Probar 3DES
3. Probar AES-256
4. Salir
[+] Seleccione una opción (1-4): 3

*** CONFIGURACIÓN PARA AES-256 ***
[+] Ingrese el texto a cifrar: Input textual para AES 256
[+] Ingrese la Clave (Recomendado 32 caracteres): Llavede32caracterespara256bitsxd
[+] Ingrese el IV (Recomendado 16 caracteres): IVde16caracteres

*** PROCESAMIENTO Y AJUSTES ***

[*] Key final utilizada (Hex): 4c6c617665646533326361726163746572657370617261323536626974737864
[*] IV final utilizado (Hex): 49566465313663617261637465726573

-----
[✓] Texto Cifrado (Base64): i0b4KVwQw6+XdUA6ce3PA4V/Tw/kiF4C9CLaQheRZZs=
[✓] Texto Descifrado:      Input textual para AES 256
-----

```

Figura 4: Output de script a comparar online.

AES Encryption / Decryption Tool

AES Encryption

Encryption Text

Input textual para AES 256

Secret Key

Llavede32caracterespara256bitsxd

Encryption Key Size

128 Bits

192 Bits

256 Bits

Encryption Mode

CBC

ECB

IV (optional)

IVde16caracteres

Output format

Base64

HEX

Encrypt

Encrypted Text

i0b4KVwQw6+XdUA6ce3PA4V/Tw/kiF4C9CLaQheRZZs=
=

Figura 5: Output online de anycrypt.com a comparar con script.

La razón de los outputs exactos se fundamenta en:

1. **Ausencia de Relleno Aleatorio:** Al ingresar la clave con sus 32 bytes exactos, no intervino la función `get_random_bytes`, haciendo que la clave en memoria local sea idéntica a la procesada por la plataforma en la nube.
2. **Alineamiento de Padding:** Ambos sistemas procesaron los bloques bajo el esquema PKCS#7, garantizando que el bloque de datos final se complete bajo la misma regla aritmética antes de realizar las multiplicaciones matriciales del algoritmo AES.

2.6. Describe la aplicabilidad del cifrado simétrico en la vida real

Un caso de uso súper común para el cifrado simétrico en la vida real es proteger bases de datos que guardan información sensible de forma local (lo que se conoce como *Data-at-Rest Encryption*), como por ejemplo registros de fichas clínicas o transacciones de cuentas bancarias. Para esto, la recomendación ideal es usar **AES-256 en modo CBC o GCM**. Como el servidor de la empresa es el único que cifra y descifra usando la misma contraseña local, no existe el problema de andar compartiendo claves con externos. Además, AES-256 es extremadamente rápido y las CPUs modernas de los servidores vienen listas con instrucciones de hardware (AES-NI) para acelerarlo, por lo que el impacto en la velocidad para los usuarios al hacer consultas en la base de datos es prácticamente imperceptible.

Si un cliente o la contraparte técnica de un proyecto no prefiriera esta recomendación y propusiera usar funciones Hash (como SHA-256) en lugar de cifrado simétrico para asegurar los datos, opinaría que están confundiendo las herramientas. Las funciones hash son unidireccionales e irreversibles por diseño, pensadas principalmente para chequear integridad (como verificar contraseñas o firmas digitales sin tener que guardar el texto original). Si aplicamos SHA-256 a los registros clínicos de los pacientes o a los saldos bancarios de los clientes, estaríamos destruyendo los datos reales para siempre porque no hay ninguna manera de dar marcha atrás y recuperar la información legible original. En cambio, para una base de datos operativa, se necesita sí o sí que el proceso sea reversible (cifrar al guardar y descifrar al consultar), una función básica que solo puede cumplir un algoritmo de cifrado simétrico.

3. Conclusiones y Comentarios

En este laboratorio se codeó y probó un script en Python 3 para implementar cifrado y descifrado simétrico por bloques (con DES, 3DES y AES-256) en modo CBC. Las pruebas que se realizaron comparando el script con la encriptación de la página online anycrypt.com/crypto resultaron en éxito, ya que el texto cifrado final en Base64 coincidió de forma exacta. Esto comprueba que, si controlamos bien los parámetros desde el inicio (usando claves e IVs que midan exactamente lo que pide cada algoritmo), la lógica de relleno PKCS#7 y el procesamiento interno funcionan idéntico sin importar el entorno o la plataforma que ejecute el código. Finalmente se describió en qué entornos se aplica el cifrado simétrico en la vida real,

siendo estos en cifrado de bases de datos que guardan información sensible de forma local (Data-at-Rest Encryption).